

# Seguimiento

Javier Dario Sanjuan De Caro

Universidad del Norte, Barranquilla, Colombia,  
[jdecaro@uninorte.edu.co](mailto:jdecaro@uninorte.edu.co)

El problema del seguimiento consiste en lograr que la salida siga la entrada de referencia lo más cerca posible. En el caso de un sistema en lazo abierto, si la planta es estable y no tiene polos en el **RHP**, teóricamente, el controlador puede ser seleccionado para cancelar la función de transferencia de la planta y reemplazarla por cualquier función de transferencia que el diseñador desee. Sin embargo, hay tres limitaciones que impiden la selección libre de la función de transferencia:

1. La función de transferencia debe ser **realizable**, lo que significa que no puede tener más ceros que polos.
2. El ingeniero debe evitar solicitar al sistema una respuesta irrealizablemente rápida. Exigir una respuesta demasiado rápida podría provocar la saturación del sistema. La saturación en un sistema de control se refiere a una condición en la que una señal dentro del sistema alcanza su valor máximo permitido o mínimo, más allá del cual no puede aumentar o disminuir. Es responsabilidad del diseñador conocer los límites del sistema de control.
3. Un diseñador podría sentirse tentado a cancelar un polo en el **RHP** de la función de transferencia de una planta. Sin embargo, dado que la planta está sujeta a cambios o existe incertidumbre en el modelo de la planta, esta práctica puede ser catastrófica, ya que la planta "real" podría tener un polo ligeramente diferente al calculado, lo que generaría una posible inestabilidad.

## 1 Ejemplo de Control de Seguimiento

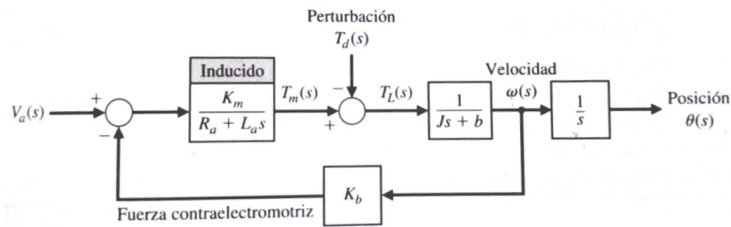


Fig. 1. Función de transferencia de un motor DC

Considere el diagrama de bloques de un motor de corriente continua presentado en la Figura 1. La función de transferencia que relaciona la velocidad angular del motor ( $\omega(s)$ ) con el voltaje aplicado ( $V_a(s)$ ) se obtiene utilizando el siguiente script en Python (symbolicMotor):

```
1 import sympy as sp
2
3 # Define las variables simbólicas
4 s = sp.symbols('s')
5 km, L, R, J, B, kb = sp.symbols('km L R J B kb')
6
7 # Define las funciones de transferencia
8 tf1 = km / (L * s + R)
```

```

9 tf2 = 1 / (J * s + B)
10
11 # Define la función de transferencia de realimentación (feedback)
12 # La función de transferencia en lazo cerrado es G / (1 + G*H), donde G es
    el sistema abierto y H el feedback
13 # En este caso, asumimos que el feedback es simplemente kb, sin dependencia
    de s para simplificar
14 motor_tf_open = tf1 * tf2 # Sistema en lazo abierto
15 motor_tf = motor_tf_open / (1 + motor_tf_open * kb)
16
17 # Simplificar la función de transferencia resultante
18 motor_tf = sp.simplify(motor_tf)
19 num, denom = motor_tf.as_numer_denom()
20 denom = sp.expand(denom)
21 motor_tf = num/denom
22 print("La función de transferencia resultante es:")
23 sp.pprint(motor_tf, use_unicode=True)

```

De esta forma, la función de transferencia del motor se obtiene de la siguiente manera:

$$\frac{\omega(s)}{V_a(s)} = \frac{k_m}{JLs^2 + (BL + JR)s + k_b k_m + BR} \quad (1)$$

Considere la lista de parámetros de un [Motor Maxon](#), como se presenta en la Tabla 1.

**Table 1.** Parámetros del motor

| Parámetro                    | Símbolo | Valor                                        |
|------------------------------|---------|----------------------------------------------|
| Resistencia de armadura      | $R$     | $2.56 \Omega$                                |
| Inductancia                  | $L$     | $8 \times 10^{-6} \text{ H}$                 |
| Constante de torque          | $k_m$   | $7.79 \times 10^{-4} \text{ N m A}^{-1}$     |
| Constante de velocidad       | $k_b$   | $7.76 \times 10^{-4} \text{ V s rad}^{-1}$   |
| Constante de amortiguamiento | $B$     | $1.46 \times 10^{-8} \text{ N m s rad}^{-1}$ |
| Inercia                      | $J$     | $1.7 \times 10^{-9} \text{ kg m}^2$          |

Considere la respuesta del sistema utilizando el siguiente código en Python (Motor.velocidad), el cual calcula la velocidad angular de salida del motor ante una entrada de paso escalón de voltaje.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import control as ctl
4
5 time = np.linspace(0, 0.1, 1000) # Time vector for all responses
6
7 # Parameters
8 R = 2.56
9 L = 0.008 * (10**-3)
10 km = 0.779 * (10**-3)
11 kb = 1/1288.05
12 B = 1.46 * (10**-8)
13 J = 0.017 * (10**-7)
14
15 # Define transfer functions
16 tf1 = ctl.tf([km], [L, R])
17 tf2 = ctl.tf([1], [J, B])
18
19 # Define the feedback transfer function

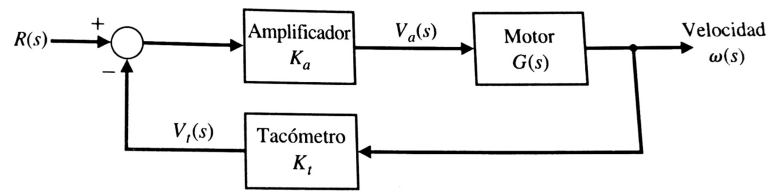
```

```

20 sys_feedback = ctl.feedback(tf1*tf2, kb)
21
22 # Compute the zeros and poles of the system
23 zeros = ctl.zero(sys_feedback)
24 poles = ctl.pole(sys_feedback)
25
26 # Compute the step response
27 t_out, response = ctl.step_response(sys_feedback, T=time)
28
29 # Create a plot for the step response
30 plt.figure(figsize=(10, 6))
31 plt.plot(t_out, response)
32 plt.title('Step Response of the Feedback System')
33 plt.xlabel('Time [s]')
34 plt.ylabel('Response')
35 plt.grid(True)
36 plt.show()
37
38 # Print the zeros and poles for reference
39 print("Zeros of the System:", zeros)
40 print("Poles of the System:", poles)
41
42 # Print the feedback system for reference
43 print("Feedback System:", sys_feedback)

```

Si uno quisiera aumentar la velocidad de respuesta del sistema, una opción sería incrementar los polos de la función de transferencia presentada en la ecuación 1. Sin embargo, para lograr eso, uno tendría que modificar físicamente el motor.



**Fig. 2.** Diagrama de bloques de un sistema de control de velocidad de un motor de corriente continua.

Consideremos ahora el caso de un sistema en lazo cerrado, como el presentado en el diagrama de bloques de la Figura 2, donde se observa que el sistema utiliza un tacómetro para generar un voltaje de retroalimentación proporcional a la velocidad. De esta forma, la función de transferencia en lazo cerrado estaría dada por:

$$\begin{aligned}
 \frac{\omega(s)}{R(s)} &= \frac{K_a G(s)}{1 + K_a K_t G(s)} \\
 &= \frac{K_a k_m}{JLs^2 + (BL + JR)s + k_b k_m + K_a K_t k_m}
 \end{aligned} \tag{2}$$

Comparemos la respuesta del sistema ante una entrada de paso escalón con la respuesta del sistema retroalimentado, para lo cual consideremos el siguiente código(motor\_comparacion\_feedback\_openloop):

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import control as ctl

```

```

4
5 # Parámetros del motor
6 R = 2.56
7 L = 0.008 * (10**-3)
8 km = 0.779 * (10**-3)
9 kb = 1/1288.05
10 B = 1.46 * (10**-8)
11 J = 0.017 * (10**-7)
12 K = 0.01 # Ganancia del controlador proporcional
13 kt = 1 # Ganancia del tacómetro
14
15 # Define transfer functions
16 tf1 = ctl.tf([km], [L, R])
17 tf2 = ctl.tf([1], [J, B])
18
19 # Sistema en lazo abierto sin controlador
20 sys_open_loop = tf1 * tf2
21 sys_open_loop = ctl.feedback(tf1*tf2, kb)
22
23 # Sistema en lazo cerrado con controlador proporcional y feedback de back
    emf
24 sys_feedback = ctl.feedback(K * sys_open_loop, kt)
25
26 # Salida del controlador
27 controller_output = ctl.feedback(K, kt * sys_open_loop)
28
29
30 # Configuración de tiempo
31 time = np.linspace(0, 0.07, 1000) # Vector de tiempo para todas las
    respuestas
32
33 # Compute the step response for open loop and closed loop
34 t_open, response_open = ctl.step_response(sys_open_loop*0.46, T=time)
35 t_closed, response_closed = ctl.step_response(sys_feedback*600, T=time)
36 t_controller, response_controller = ctl.step_response(controller_output
    *600, T=time)
37
38 # Create a figure for the subplots
39 fig, (ax1, ax2) = plt.subplots(2, figsize=(8, 6))
40
41 # Create a plot for the step responses
42
43 ax1.plot(t_open, response_open, label='Open-Loop Response')
44 ax1.plot(t_closed, response_closed, label='Closed-Loop Response')
45 ax1.set_title('Step Response of the Motor System')
46 ax1.set_xlabel('Time [s]')
47 ax1.set_ylabel('Angular Velocity [rad/s]')
48 ax1.grid(True)
49 ax1.legend()
50
51 # Plot the controller's response
52 ax2.plot(t_controller, response_controller)
53 ax2.set_title('Controller's Response')
54 ax2.set_xlabel('Time [s]')
55 ax2.set_ylabel('Response')
56 ax2.grid(True)

```

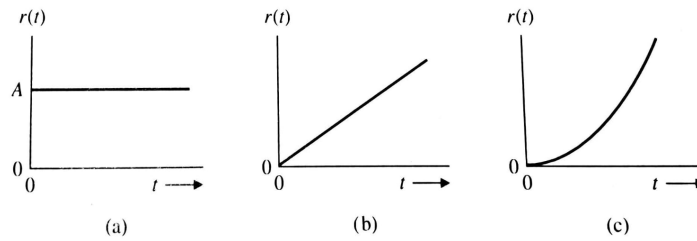
```

57
58 plt.tight_layout()
59 plt.show()

```

Note que la respuesta del sistema es mucho más rápida gracias a la utilización del controlador y, lo más importante, esto se logró sin tener que cambiar ningún parámetro físico del sistema. Por otro lado, el código también presenta la gráfica de voltaje de salida del controlador. De esta forma, se puede ver que la velocidad del sistema se incrementó aumentando el voltaje de salida del controlador. Como ingeniero, debe estar muy pendiente de este tipo de salidas, debido a que se puede presentar el caso en el que el sistema se sature si la respuesta requerida es demasiado rápida, o que se alcance un voltaje muy alto que puede dañar el motor. Otra característica que se observa es la presencia de error; note que se requería llegar a un valor de velocidad angular de  $600 \text{ rad s}^{-1}$ , pero se obtuvo un valor un poco menor de  $558 \text{ rad s}^{-1}$ . De este problema hablaremos en la sección siguiente.

## 2 Control de Seguimiento para Entradas Polinomiales



**Fig. 3.** Señales de entrada de prueba: a) escalón, b) rampa y c) parabólica

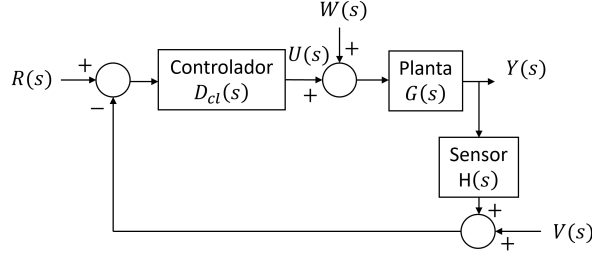
**Table 2.** Señales de Entrada de Prueba y sus Representaciones

| Señal de Prueba | Función en el Tiempo | Transformada de Laplace |
|-----------------|----------------------|-------------------------|
| Escalón         | $Au(t)$              | $A/s$                   |
| Rampa           | $At$                 | $A/s^2$                 |
| Parabólica      | $At^2$               | $2A/s^3$                |

En el caso general de seguimiento, la entrada de referencia suele ser o se aproxima a un polinomio de grado bajo. Por ejemplo, en el control de posición de un elevador, se utiliza una rampa como función de referencia, como se muestra en la Figura 3 y en la Tabla 2. Esta función hará que el elevador se mueva a una velocidad constante hasta llegar al siguiente piso. En casos poco comunes, la entrada de referencia puede ser aproximada, durante un intervalo sustancial de tiempo, a una aceleración constante. La consideración de este tipo de ejemplos nos lleva a plantearnos las siguientes preguntas: Dada una función de transferencia, ¿esta puede seguir todo tipo de trayectorias? Y si no se puede, ¿qué características debe satisfacer un modelo para que sí se pueda? La respuesta a estas preguntas las exploraremos en la sección siguiente.

### 2.1 Tipos de Sistemas de Seguimiento

En el diagrama de bloques presentado en la figura 4, se obtiene la siguiente función de transferencia para el error en función de la referencia:



**Fig. 4.** Diagrama de bloques de un sistema en lazo cerrado (**closed loop**)

$$E(s) = \frac{1}{1 + D_{cl}GH(s)} R(s) \quad (3)$$

Para considerar entradas polinomiales, se define  $r(t) = \frac{u(t)t^k}{k!}$ , como una entrada polinomial general, cuya Transformada de Laplace es  $R = \frac{1}{s^{k+1}}$ . Se establece que cuando  $k = 0$  corresponde a una entrada de paso escalon,  $k = 1$  a una entrada de rampa, y  $k = 2$  a una entrada parabólica. Así, asumiendo  $H(s) = 1$  y utilizando el teorema del valor final, se plantea:

$$\begin{aligned} \lim_{t \rightarrow \infty} e(t) &= e_{ss} = \lim_{s \rightarrow 0} sE(s) \\ &= \lim_{s \rightarrow 0} \frac{1}{1 + GD_{cl}} R(s) \\ &= \lim_{s \rightarrow 0} \frac{1}{1 + GD_{cl}} \frac{1}{s^{k+1}} \end{aligned} \quad (4)$$

Consideremos un sistema para el cual  $GD_{cl}$  no tiene un polo en el origen y  $R(s) = \frac{1}{s}$ . Así,  $r(t)$  es un polinomio de grado 0. En este caso, el error se reduce a:

$$\begin{aligned} e_{ss} &= \lim_{s \rightarrow 0} s \frac{1}{GD_{cl}} \frac{1}{s} \\ e_{ss} &= \frac{1}{1 + GD_{cl}(0)} \end{aligned} \quad (5)$$

Este sistema se define como **Tipo 0** (porque tiene 0 polos en el origen). Note que la función de transferencia produce un error  $e_{ss} \neq 0$  y que si la función de entrada fuera un polinomio de grado 1, el error resultante crecería sin límites. Un polinomio de grado 0 es el grado más alto que un sistema de **Tipo 0** puede seguir sin error en estado estacionario. Para generalizar el análisis, dividamos la función de transferencia  $GD_{cl}(s)$  como una función de transferencia  $K_n(s)$  sin polos en el origen, y una función de transferencia  $s^n$ , como se muestra:

$$GD_{cl}(s) = \frac{K_n(s)}{s^n} \quad (6)$$

Así, si la función de transferencia es de tipo 0,  $n = 0$ . Sustituyendo esta expresión en la ecuación (4) se obtiene:

$$e_{ss} = \lim_{s \rightarrow 0} \frac{s^n}{s^n + K_n(s)} \frac{1}{s^k} \quad (7)$$

De esta ecuación, se puede observar que si  $n > k$  entonces  $e = 0$  y que si  $n < k$  entonces  $e \rightarrow \infty$ . Por otro lado, si  $n = k = 0$ , entonces  $e_{ss} = \frac{1}{1 + K_0}$ , y si  $n = k \neq 0$ , entonces  $e_{ss} = \frac{1}{K_n}$ . Así, si  $n = k = 0$ , la entrada es un polinomio de grado 0, conocido como un paso escalón, y la

constante  $K_0$  es llamada la **constante de posición** y se escribe  $K_p$ . Si  $n = k = 1$ , el sistema es de **tipo 1** y la entrada es un polinomio de grado 1, también conocido como entrada de rampa, donde la constante  $K_1$  es llamada **constante de velocidad** y se escribe como  $K_v$ . De forma similar, se pueden definir sistemas de **Tipo 2** o superior, clasificando el error para diferentes tipos de sistemas como el presentado en la tabla 3.

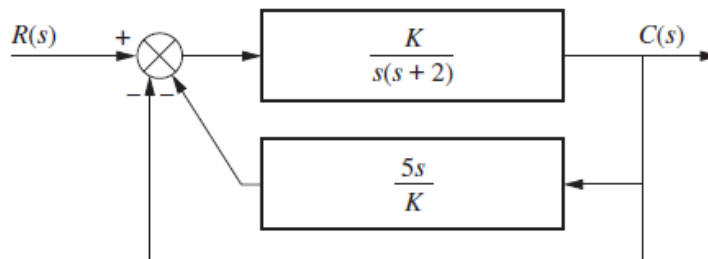
**Table 3.** Error como función del tipo de sistema

| Tipo de Sistema | Paso Escalón (Posición) | Rampa (Velocidad) | Parábola (Aceleración) |
|-----------------|-------------------------|-------------------|------------------------|
| Tipo 0          | $\frac{1}{1+K_p}$       | $\infty$          | $\infty$               |
| Tipo 1          | 0                       | $\frac{1}{K_v}$   | $\infty$               |
| Tipo 2          | 0                       | 0                 | $\frac{1}{K_a}$        |

La definición de tipo de sistema nos ayuda a identificar la habilidad de un sistema para seguir polinomios. En una estructura de retroalimentación unitaria, si los parámetros del proceso cambian sin remover los polos en el origen en un sistema de **Tipo 1**, la constante de velocidad cambiará, pero el sistema tendrá cero error en estado estable. Una conclusión similar se puede tener para sistemas de tipo superior. De esta forma, podemos decir que los tipos de sistema son **robustos** con respecto a cambios en los parámetros en la estructura de lazo cerrado con retroalimentación unitaria.

### 3 Problemas

1. Se muestra un diagrama de bloques de un sistema de control en la figura 5. Diseñe el valor de  $K$  tal que para una entrada  $100tu(t)$ , se tenga un error de 0.01 en el estado estable.



**Fig. 5.** Sistema de lazo cerrado

### References

1. FRANKLIN, Gene F., et al. Feedback control of dynamic systems. Upper Saddle River: Prentice hall, 2002.
2. NISE, Norman S. Control systems engineering. John Wiley & Sons, 2020.
3. DORF, Richard C., et al. Sistemas de control moderno. Pearson Educación, 2005.
4. Umez-Eronini, Eronini, and Enriquecotr Palos. Dinámica de sistemas y control. Thomson Learning. 2001.